# Lab 10 : Explainable Zero-Trust Code Analysis and Behavioural Classification Pipeline

## Lab Learning Outcomes

By the end of the lab, students should be able to:

- understand how to implement the coursework
- integrate the sandbox into a Compose-based zero-trust monitoring architecture,
- collect and persist runtime evidence in MongoDB,
- correlate telemetry and classifications using `run_id`,
- compare system-provided classifications with independently derived reasoning,
- maintain an auditable trail linking observation, explanation, policy, and result,
- distinguish between automated labels and justified technical conclusions.

## Background information

Modern software development increasingly relies on Generative AI tools that can quickly produce working code. Although such tools improve productivity, they also create a risk that users may copy and execute generated scripts without understanding their behaviour or system impact. In educational environments this can lead to accidental “unintentional hacking”, where code performs unsafe actions such as network communication, subprocess execution, filesystem modification, credential access, or excessive resource consumption. A zero-trust approach

therefore treats all candidate code as untrusted by default and investigates behaviour through controlled execution and telemetry rather than through assumptions about intent.

The earlier labs in the module introduce the individual components needed to analyse and manage this risk. Lab 5 establishes threat modelling and static code analysis using Abstract Syntax Trees to detect risky primitives. Lab 6 introduces sandboxed execution and runtime telemetry capture so that behaviour can be observed safely. Labs 2, 3, and 4 provide computational-intelligence methods (swarm optimisation, evolutionary computation, and reinforcement learning) that can be used to explore input spaces, configuration parameters, and adaptive probing strategies. Lab 7 then introduces a multi-agent monitoring architecture with adaptive escalation, while Labs 8 and 9 extend the system to distributed microservice pipelines and CI/CD-driven optimisation workflows.

Lab 10 serves as the integration stage that brings these capabilities together into a single explainable and auditable system. Students construct an **Adaptive Code Safety Harness** that combines static analysis, sandboxed execution, telemetry monitoring, computational-intelligence search methods, and multi-agent evidence fusion within a zero-trust monitoring architecture. The system must treat candidate code as untrusted, analyse its structure, execute it in a constrained environment, observe behavioural signals, and adaptively probe the execution to uncover suspicious activity. Importantly, decisions about risk must be justified using recorded evidence rather than opaque automated outputs. The final output of the system is an explainable risk report that links observed behaviour, probe results, policy checks, and mitigation guidance. Lab 10 therefore connects the entire coursework: the threat model and static features from Lab 5, the sandbox and telemetry mechanisms from Lab 6, the monitoring and escalation architecture from Lab 7, the distributed pipeline concepts from Lab 8, and the optimisation strategies from Lab 9. By integrating these components, Lab 10 demonstrates how computational intelligence methods can be applied responsibly to analyse untrusted code, produce transparent security assessments, and support safe development practices.

# Introduction to MongoDB

MongoDB is a document-oriented NoSQL database designed to store and manage semi-structured data efficiently. Unlike traditional relational databases that rely on fixed schemas and tables, MongoDB stores information as flexible JSON-like documents (BSON). Such flexibility makes it well suited for systems where the structure of stored data may evolve over time or where records contain nested and heterogeneous information. In monitoring and security-analysis pipelines, telemetry snapshots, model outputs, configuration states, and investigation reports rarely follow a single rigid schema. A document database therefore allows each investigation record to capture rich contextual information without forcing strict relational constraints.

Within the context of this lab, MongoDB functions as the **persistent audit store** for the Adaptive Code Safety Harness. Each execution run can generate multiple types of data: static analysis findings, telemetry observations, sandbox metadata, model assessments, risk scores, and policy decisions. Instead of treating these signals as temporary logs, the harness stores them as structured documents tied to a unique run identifier. A single run record can therefore contain the API identifier, the `run_id`, the telemetry events observed during execution, the classification returned by the sandbox, and the system's own reasoning. Storing this information centrally ensures that decisions made by the system can be reconstructed and reviewed later.

Explainability improves significantly when reasoning artefacts are preserved alongside raw observations. In this lab, Ollama provides the local language model that interprets telemetry signals and generates structured reasoning about potential risks, suspicious behaviour, and mitigation guidance. However, model outputs alone do not provide reliable explanations unless they are linked to the evidence that produced them. MongoDB therefore stores both the evidence and the reasoning results in the same audit trail. A typical document may include the

telemetry snapshot, the model-generated summary, a confidence estimate, the triggered risk indicators, and the final risk score. This approach allows investigators to trace how the model's interpretation relates to the underlying signals.

The combined use of MongoDB and Ollama therefore creates an **explainable monitoring pipeline**. Ollama performs contextual interpretation of telemetry and behavioural patterns, while MongoDB records every observation, inference, and decision in a persistent and queryable form. Because the stored documents retain both raw data and interpreted results, students can later analyse why a classification was produced, compare multiple runs, evaluate alternative configurations, or audit whether policies were applied correctly. In practice, MongoDB provides the transparency and traceability layer, while Ollama provides the reasoning layer, together enabling a monitoring system that is both intelligent and auditable.

## How does it work?

MongoDB is a document-oriented NoSQL database that stores data in collections of flexible JSON-like documents. Each document is internally represented using BSON (Binary JSON), which allows efficient storage and indexing of nested structures. Unlike relational databases that require predefined tables and rigid schemas, MongoDB allows documents in the same collection to contain different fields. Such flexibility is particularly useful in telemetry and monitoring systems where records may include varying structures such as runtime metrics, model outputs, event logs, and configuration metadata. A MongoDB database contains multiple **collections**, and each collection contains **documents**. A document is essentially a structured object composed of key–value pairs. Each document automatically includes a unique identifier called `_id`. For example, a monitoring system that records sandbox execution runs might store documents such as:

```
{
  "_id": "run_01",
  "api_id": "api07",
  "classification": "benign",
  "potentially_harmful": false,
  "cpu_usage": 18.3,
  "network_events": 0,
  "timestamp": "2026-03-15T10:02:11Z"
}
```

Although MongoDB does not require a strict schema, it is good practice to define a **logical schema** so that documents follow a predictable structure. For example, a collection called `sandbox_runs` might follow a schema such as:

```
sandbox_runs
 ├── run_id
 ├── api_id
 ├── classification
 ├── telemetry
 │    ├── cpu
 │    ├── network
 │    ├── filesystem
 │    └── threads
 ├── model_analysis
 │    ├── summary
 │    ├── confidence
 │    └── explanation
 └── timestamp
```

In MongoDB, creating a collection is simple. Collections are typically created automatically when the first document is inserted, but they can also be created explicitly. Example of creating a collection:

```
use lab10_audit

db.createCollection("sandbox_runs")
```

To insert a document into the collection:

```
db.sandbox_runs.insertOne({
  run_id: "run_001",
  api_id: "api03",
  classification: "suspicious",
  potentially_harmful: true,
  telemetry: {
    cpu: 72.5,
    network_attempts: 3,
    file_changes: 2
  },
  timestamp: new Date()
})
```

Multiple documents can also be inserted at once:

```
db.sandbox_runs.insertMany([
  {
    run_id: "run_002",
    api_id: "api04",
    classification: "benign",
    cpu_usage: 12.4
  },
  {
    run_id: "run_003",
    api_id: "api05",
    classification: "potentially_harmful",
    cpu_usage: 95.2
  }
])
```

Query operations allow retrieval of documents that match specific conditions. The simplest query returns all documents in a collection:

```
db.sandbox_runs.find()
```

To search for documents with a specific classification:

```
db.sandbox_runs.find({ classification: "benign" })
```

More advanced queries can filter by multiple conditions:

```
db.sandbox_runs.find({
  cpu_usage: { $gt: 80 },
  potentially_harmful: true
})
```

Projection can be used to return only selected fields:

```
db.sandbox_runs.find(
  { classification: "suspicious" },
  { api_id: 1, cpu_usage: 1 }
)
```

Updating documents is performed using update operations. For example, to update the classification of a run:

```
db.sandbox_runs.updateOne(
  { run_id: "run_001" },
  { $set: { classification: "benign", potentially_harmful: false }
}
)
```

Multiple documents can be updated simultaneously:

```
db.sandbox_runs.updateMany(
  { cpu_usage: { $gt: 90 } },
  { $set: { classification: "resource_abuse" } }
)
```

Fields can also be incremented or modified dynamically:

```
db.sandbox_runs.updateOne(
  { run_id: "run_002" },
  { $inc: { network_attempts: 1 } }
)
```

Deleting documents removes records from a collection. For example:

Delete a specific run:

```
db.sandbox_runs.deleteOne({ run_id: "run_003" })
```

Delete all benign runs:

```
db.sandbox_runs.deleteMany({ classification: "benign" })
```

In the context of the lab system, these operations support explainability and auditing. Telemetry observations can be inserted as documents, model interpretations generated by Ollama can be stored as additional fields, and policy decisions can be updated as the monitoring pipeline evolves. Queries allow

investigators to analyse past runs, compare classifications across experiments, and identify patterns such as frequent network access attempts or abnormal resource usage. By persisting both evidence and interpretation, MongoDB enables the system to maintain a transparent record of how each risk decision was reached.

# Lab 10: Bringing all together

**This lab contributes to your assessed coursework. Ensure that you save your completed solution in your private CWK repository.**

The system will interact with the sandbox as an **external investigation environment**, where the internal execution mechanisms are not directly accessible to students. Instead of implementing the sandbox itself, students build an **analysis and monitoring harness** that observes and interprets behaviour through the sandbox's exposed telemetry and classification APIs.

The system will therefore:

- treat candidate code **executed inside the sandbox container** as untrusted by default
- interact with the sandbox **exclusively through its exposed REST endpoints** rather than inspecting internal scripts or processes
- collect **static analysis signals** from the candidate code before execution
- gather **dynamic behavioural signals** from sandbox telemetry endpoints (network activity, filesystem events, process behaviour, and resource usage)
- analyse the observed behaviour using **Computational Intelligence (CI) methods** such as evolutionary probing, swarm-based configuration exploration, or reinforcement-learning policies
- generate **structured explanations and risk reports** that describe the evidence supporting each classification
- store investigation evidence, telemetry snapshots, and reasoning outputs in **MongoDB** to create a persistent and auditable record of the analysis

- use **local LLM reasoning through Ollama** to interpret telemetry patterns and produce human-readable explanations of potential risks
- correlate collected evidence with the **sandbox classification endpoints** (`/results/last` and `/results`) in order to compare system-generated labels with the harness's independent analysis

**NOTE that you cannot modify or access the sandbox internals.** Instead, they develop an **external monitoring and reasoning system** that observes execution behaviour, analyses risk signals, and produces explainable security assessments based on telemetry and evidence.

## 1. System Architecture

The full lab must be organised as a **Docker Compose stack** so that all services can be started, stopped, and reproduced consistently. The sandbox must be included as a service in `docker-compose.yml`, and students must interact with it only through its exposed APIs. The **comp40771 container** is the main working environment where notebooks, Python scripts, and orchestration code are executed. Other services provide monitoring, reasoning, and storage. A recommended project structure is shown below.

```
.
├── configs
│   ├── ci_config.json
│   ├── risk_config.json
│   └── sandbox_config.json
├── docker-compose.yml
├── mongo-init
│   └── init.js
├── monitor
│   ├── agents
│   │   ├── filesystem_agent.py
│   │   ├── network_agent.py
│   │   ├── process_agent.py
│   │   ├── __pycache__
│   │   │   ├── filesystem_agent.cpython-311.pyc
│   │   │   ├── network_agent.cpython-311.pyc
│   │   │   ├── process_agent.cpython-311.pyc
│   │   │   └── resource_agent.cpython-311.pyc
│   │   └── resource_agent.py
│   ├── configs
│   ├── Dockerfile
│   ├── monitor.py
│   ├── reports
│   ├── requirements.txt
│   └── utils
│       ├── __pycache__
│       │   ├── reporting.cpython-311.pyc
│       │   ├── scoring.cpython-311.pyc
│       │   ├── telemetry.cpython-311.pyc
│       │   └── telemetry.cpython-312.pyc
```

```
│       ├── reporting.py
│       ├── scoring.py
│       └── telemetry.py
├── notebooks
│   └── comp40771_lab.ipynb
├── ollama
│   ├── Dockerfile
│   └── entrypoint.sh
├── README.md
└── reports
```

Table 1: Core components

| Service | Purpose |
|---|---|
| **sandbox** | Executes candidate scripts and exposes monitoring and classification APIs |
| **monitor** | Collects telemetry from the sandbox and orchestrates behavioural analysis |
| **ollama** | Runs the local language model used for reasoning and explanation |
| **mongodb** | Stores telemetry records, classifications, and investigation reports |
| **comp40771 container** | Primary development environment where notebooks and scripts run |

| | |
|---|---|
| **optional containers** | Additional services such as dashboards, message brokers, CI tools, or experiment managers |

Recommended architecture:

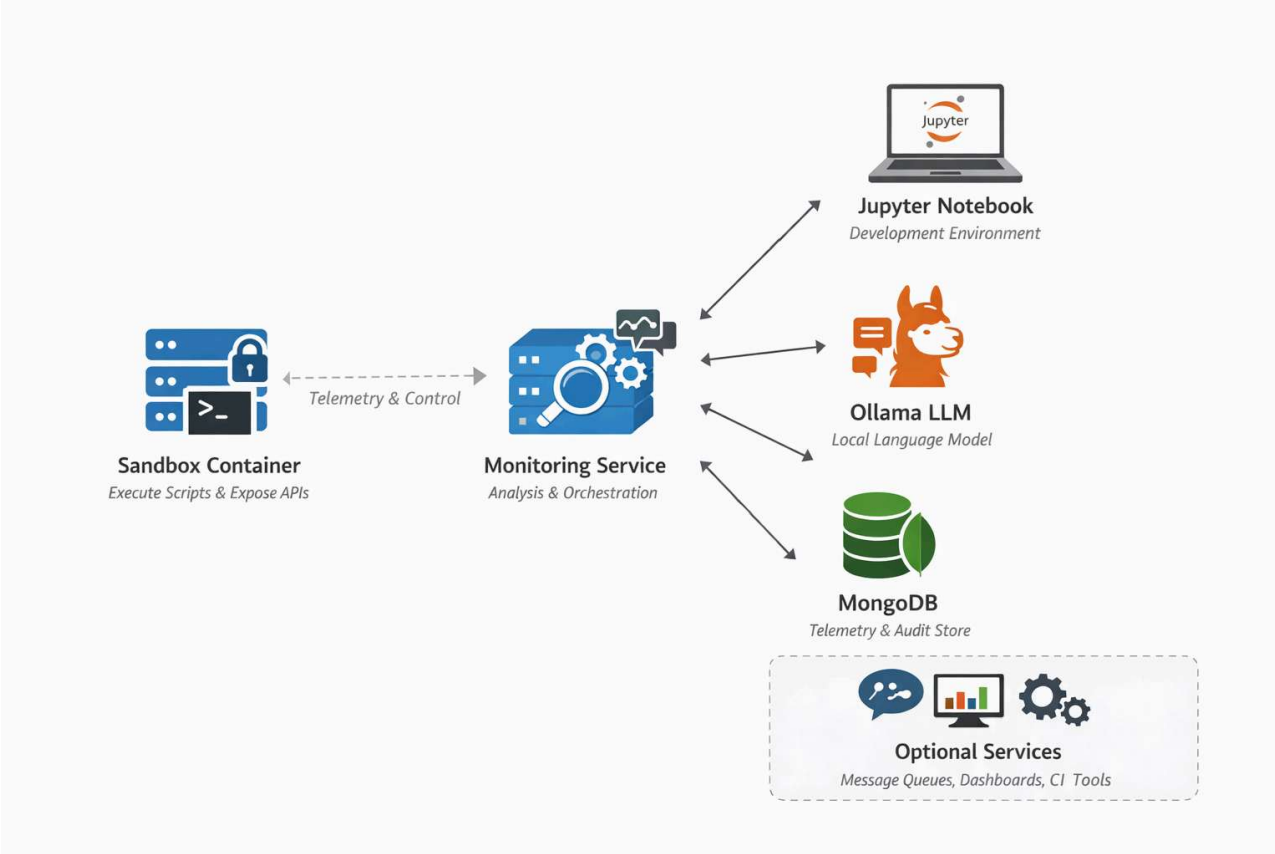


Figure 1: recommended architecture

## 2. Required Docker Compose Environment

The entire lab/cwk environment must run using **Docker Compose**, ensuring that all services are deployed in a consistent and reproducible way inside the module container infrastructure. The sandbox must be integrated as a **service in the Compose stack** rather than executed manually using `docker run`. Students must

interact with the sandbox only through its exposed APIs. You are allowed to extend the architecture with **additional containers** where this improves monitoring, analysis, automation, or reproducibility.

**Forking and cloning the project from the git repository**

Step 1: Login into https://olympus.ntu.ac.uk/ [https://olympus.ntu.ac.uk/] using your NTU credentials.

Step 2: Goto the https://olympus.ntu.ac.uk/comp40771/comp40771_lab10 [https://olympus.ntu.ac.uk/comp40771/comp40771_lab10] and fork it

Step 3: Goto your account (e.g. N123456/comp40771_lab8) hit the button **Code** and copy the repository url.

Step 4: **Clone or download into your HOST machine**.

Step 5: Clone the repository

**Jupyter lab terminal**

```
git clone https://olympus.ntu.ac.uk/YOUR-STUDENT-NUMBER/comp40771_lab10
```

where `YOUR-STUDENT-NUMBER` is your student number. Enter your student ID in the field Username and your NTU password in the field Password.

**Use docker compose to build the containers.**

**PowerShell/Mac/Linux terminal**

```
docker compose up -d --build
```

Or if you have already built the containers

**PowerShell/Mac/Linux terminal**

```
docker compose up
```

## 3. Run and analyse the code

Open a tab in your favourite browser (e.g. Chrome, Firefox, Edge, Safari, etc.), enter **http://localhost:8081/lab** and open a terminal. **Do not clone the repository inside the container!** Run and study the **notebooks/comp40771_lab10.ipynb.**

## 4. Sandbox Investigation Workflow

Each investigation follows a controlled execution cycle. **NOTE** that the `curl` command is to run from the jupyter terminal.

**Step 1: Start telemetry**

Telemetry must be activated before running any API scenario.

```
POST /telemetry/start
```

Example:

```
curl -X POST http://localhost:8030/telemetry/start \
-H "Content-Type: application/json" \
-d '{}'
```

**Step 2: Start an API scenario**

Each API scenario simulates behaviour that must be investigated.

```
POST /apis/{api_id}/start
```

Example:

```
curl -X POST http://localhost:8030/apis/api07/start
```

The sandbox returns metadata describing the run.

Example response:

```
{
  "api_id": "api07",
  "run_id": "b6a2b2b7-1b9f-4f18-b49f-2b2c3f7e20c7"
}
```

The **run_id uniquely identifies the execution instance** and should be recorded for audit purposes.

**Step 3: Monitor behaviour**

Telemetry endpoints expose runtime signals from the sandbox.

Examples:

```
GET /status
GET /monitor/network
GET /monitor/files
GET /monitor/cpu
GET /monitor/threads
GET /monitor/storage
GET /telemetry/events/recent
```

Example:

```
curl http://localhost:8030/monitor/network
```

These endpoints provide the signals used to infer behavioural risk.

**Step 4: Retrieve classification results**

The sandbox produces a classification after each run.

Latest run:

```
GET /results/last
```

Example response:

```
{
  "run_id": "b6a2b2b7-1b9f-4f18-b49f-2b2c3f7e20c7",
  "api_id": "api07",
  "classification": "benign",
  "potentially_harmful": false,
  "started_at": "2026-03-15T10:02:11Z"
}
```

Full session history:

```
GET /results
```

Example:

```
curl http://localhost:8030/results
```

Students should correlate these results with their own evidence-based analysis.

**Step 5 – Stop scenario**

Once analysis is complete, the API run should be terminated.

```
POST /apis/stop
```

Example:

```
curl -X POST http://localhost:8030/apis/stop
```

## 5. Monitoring and Evidence Collection Pipeline

The monitoring pipeline is responsible for collecting behavioural evidence from the sandbox and preparing it for analysis. The system operates through a **sense->reason->classify workflow.**

**Sense phase: Collect telemetry**

The monitoring service retrieves runtime signals exposed by the sandbox. Typical telemetry sources include:

| Signal | Endpoint |
|---|---|
| CPU usage | `/monitor/cpu` |
| network activity | `/monitor/network` |
| filesystem events | `/monitor/files` |
| thread behaviour | `/monitor/threads` |
| storage activity | `/monitor/storage` |
| runtime events | `/telemetry/events/recent` |

Example telemetry collection:

```
def collect_telemetry():
    return {
        "cpu": requests.get(f"{SANDBOX}/monitor/cpu").json(),
        "network": requests.get(f"
{SANDBOX}/monitor/network").json(),
        "files": requests.get(f"{SANDBOX}/monitor/files").json(),
        "threads": requests.get(f"
{SANDBOX}/monitor/threads").json(),
        "storage": requests.get(f"
{SANDBOX}/monitor/storage").json()
    }
```

These signals form the **primary behavioural evidence** used for classification.

The `comp40771` container serves as the primary development environment for this lab. All notebooks, configuration files, and monitoring scripts can be edited and executed from within this container. The `monitor` service runs the behavioural investigation pipeline, while the `comp40771` container is used to modify and test its code. Because the `docker-compose.yml` file mounts the `monitor` directory into both containers, the source code is shared. Any modification made inside the `comp40771` container is immediately visible inside the running `monitor` container. This allows students to develop and debug the monitoring logic interactively.

### A. Start the full environment

From the root project directory, start all services:

```
docker compose up -d --build
```

This command builds the containers (if necessary) and launches the following services:

- `comp40771_container` – development environment
- `sandbox` – execution environment for candidate scripts
- `monitor` – monitoring and classification service
- `ollama` – local language model service
- `mongodb` – audit database

To verify that the containers are running:

```
docker ps
```

**B. Enter the `comp40771` container**

Open a terminal inside the development container:

```
docker exec -it comp40771_container bash
```

Inside the container, navigate to the monitor source code:

```
cd /opt/data/monitor
```

This directory corresponds to the `monitor/` folder in the project structure.

**C. Modify the monitoring code**

Students can edit the monitoring components directly from the `comp40771` container.

Key files include:

```
monitor/
├── monitor.py
├── agents/
│   ├── filesystem_agent.py
│   ├── network_agent.py
│   ├── process_agent.py
│   └── resource_agent.py
└── utils/
    ├── telemetry.py
    ├── scoring.py
    └── reporting.py
```

Example modifications might include:

- adjusting risk scoring logic in `scoring.py`
- modifying telemetry processing in `telemetry.py`
- adding new indicators in one of the agent modules
- improving classification explanations in `reporting.py`

Files can be edited using terminal editors such as:

```
nano monitor.py
```

Alternatively, students can open the Jupyter interface provided by the container:

http://localhost:8081/lab [http://localhost:8081/lab]

and edit files directly in the browser.

**D. Run the monitor manually for testing**

Before restarting the `monitor` service, it is useful to test the script interactively.

Set the required environment variables:

```
export SANDBOX_BASE=http://sandbox:8000
export OLLAMA_BASE=http://ollama:11434
export MONGO_URI=mongodb://mongodb:27017
export MONGO_DB=lab10_audit
export DEFAULT_API_ID=api01
export POLL_INTERVAL=3
export MONITOR_DURATION=10
export RISK_CONFIG=/opt/data/configs/risk_config.json
```

Then execute the monitoring script:

```
python monitor.py
```

Running the script manually allows students to observe errors, debug behaviour, and inspect telemetry output directly.

**E. Restart the monitor container**

After modifying the code, restart the monitor service so that the running container executes the updated script.

```
docker compose restart monitor
```

If dependencies or the Dockerfile were changed, rebuild the service:

```
docker compose up -d --build monitor
```

**F. Inspect monitor logs**

To observe the behaviour of the monitoring pipeline:

```
docker compose logs -f monitor
```

Typical log output includes:

- sandbox run identifiers
- telemetry collection events
- classification decisions
- MongoDB storage operations

**G. Run commands inside the monitor container**

If deeper debugging is required, a shell can be opened inside the `monitor` container:

```
docker exec -it monitor bash
```

Navigate to the application directory:

```
cd /app
python monitor.py
```

### H. Important networking note

When containers communicate with each other, they must use **Docker service names**, not `localhost`. Correct internal addresses:

```
sandbox → http://sandbox:8000
ollama → http://ollama:11434
mongodb → mongodb://mongodb:27017
```

If testing from the host machine instead of inside containers, the exposed ports should be used:

```
sandbox → http://localhost:8030 [http://localhost:8030]
ollama → http://localhost:11434 [http://localhost:11434]
mongodb → mongodb://localhost:27017 [mongodb://localhost:27017]
```

### I. Recommended development workflow

A typical workflow during the lab is:

```
docker compose up -d --build
docker exec -it comp40771_container bash
cd /opt/data/monitor

# modify files
nano monitor.py

# test script manually
python monitor.py

# restart monitor container
exit
docker compose restart monitor

# inspect logs
docker compose logs -f monitor
```

## 6. Behaviour Analysis

After telemetry is collected, the system analyses signals to identify patterns associated with risky behaviour. Possible behavioural indicators include:

| Behaviour | Interpretation |
| --- | --- |
| repeated outbound network attempts | possible data exfiltration |
| unexpected file writes | filesystem modification |
| abnormal thread activity | suspicious execution behaviour |

| Behaviour | Interpretation |
| --- | --- |
| excessive CPU usage | resource abuse |
| access to decoy resources | potential malicious behaviour |

Signals are converted into **structured features** used by the classification pipeline. Example feature vector:

```
{
  "network_attempts": 3,
  "file_changes": 2,
  "cpu_peak": 87,
  "thread_count": 6
}
```

These features can then be used for scoring, CI probing, or model interpretation.

## 7. Behaviour Classification

The system assigns a **classification label** based on the observed evidence. Typical classification labels include:

```
benign
suspicious
potentially_harmful
inconclusive
```

The classification should be supported by multiple sources of evidence, including:

- telemetry signals
- static analysis findings
- CI-generated probe outcomes
- sandbox result endpoints
- model reasoning
- deterministic scoring rules

The classification output should include:

- classification label
- confidence score
- contributing indicators
- explanation of the decision

Example classification result:

```
{
  "classification": "suspicious",
  "confidence": 0.74,
  "indicators": [
    "network_attempts",
    "filesystem_changes"
  ],
  "explanation": "Repeated outbound network connections combined
with filesystem writes."
}
```

## 8. CI-Driven Probing

Computational Intelligence techniques are used to explore behaviour space and uncover risky actions. These techniques generate **probes**, which are variations of inputs or execution contexts designed to trigger suspicious behaviour.

**Evolutionary probing**

Evolutionary algorithms can generate input variations that maximise behavioural risk signals. Example objective:

```
maximize(network_attempts + filesystem_changes + cpu_usage)
```

Typical evolutionary configuration:

```
{
  "population": 30,
  "generations": 20,
  "mutation_rate": 0.1
}
```

Evolutionary probing should demonstrate improved behaviour discovery compared to random probing.

**Swarm configuration search**

Swarm algorithms explore sandbox configuration parameters to identify minimal safe permissions. Example parameters:

```
execution timeout
CPU limit
filesystem permissions
environment variables
network allowlists
```

Goal:

```
minimal permissions while allowing benign tasks
```

**Reinforcement learning probing**

A reinforcement learning agent can select which probe to execute next based on observed signals. Objective:

```
minimise time-to-decision while maximising classification
confidence
```

The agent learns which probes reveal behavioural patterns more efficiently.

## 9. Multi-Container Agent Architecture (No action required)

The monitoring system should follow a **multi-container architecture**, where different containers perform specialised monitoring and analysis tasks. Each container acts as an **agent responsible for a specific function**, allowing the investigation pipeline to be modular, scalable, and easier to extend. Instead of implementing all monitoring logic in a single service, responsibilities are distributed across multiple containers. Each agent container focuses on analysing a particular behavioural channel or performing a specific analysis task.

Example architecture:

```
docker-compose stack
│
├── comp40771_container
│
├── sandbox
│
├── telemetry_agent
│
├── filesystem_agent
│
├── network_agent
│
├── resource_agent
│
├── ci_probe_agent
│
├── reasoning_agent (Ollama)
│
└── mongodb
```

In this architecture:

| Agent Container | Responsibility |
| --- | --- |
| **telemetry_agent** | Collects telemetry from sandbox endpoints |
| **filesystem_agent** | Analyses file access and filesystem events |
| **network_agent** | Detects outbound network attempts or unusual communication patterns |

| Agent Container | Responsibility |
| --- | --- |
| **resource_agent** | Monitors CPU, memory, and thread activity |
| **ci_probe_agent** | Generates and executes CI-driven probes (GA, PSO, RL) |
| **reasoning_agent** | Uses Ollama to generate explanations from observed evidence |
| **mongodb** | Stores investigation evidence and classification results |

Each agent container processes telemetry relevant to its domain and produces **structured observations**. These observations are then aggregated by the monitoring logic to produce a unified behavioural representation for the run. Example aggregated observation:

```
{
  "run_id": "c42f98",
  "network_agent": {
    "outbound_connections": 3
  },
  "filesystem_agent": {
    "file_writes": 2
  },
  "resource_agent": {
    "cpu_peak": 87
  }
}
```

This architecture provides several advantages:

- **Separation of concerns** – each container focuses on a specific task.
- **Scalability** – additional analysis agents can be added easily.
- **Reproducibility** – all components are defined in `docker-compose.yml`.
- **Clear evidence pipeline** – telemetry signals are processed independently and then fused for classification.

The final classification decision is therefore based on **combined evidence generated by multiple specialised agents running as separate containers**.

## 15. Expected Outcome

The completed system should demonstrate:

- telemetry-driven behavioural investigation
- CI-driven probing strategies
- structured behavioural feature extraction
- explainable classification decisions
- persistent evidence storage
- reproducible experimentation

The final outcome should show how **Computational Intelligence methods can be used to investigate untrusted code behaviour and produce transparent, evidence-based risk classifications within a zero-trust monitoring environment**. The final output should include the classification label, confidence or score, contributing factors, and recommended mitigation or least-privilege guidance.

## Stop the containers

Let's stop the containers:

**PowerShell/Mac/Linux terminal**

```
docker compose down
```

---

Go to top